



DACEFyN

Departamento Académico de Ciencias
Exactas, Físicas y Naturales

UNIDAD 2: ALGORITMOS FUNDAMENTALES

Backend de Plataforma de Streaming de Musica

Integrante 1: _____

Legajo: _____

Integrante 2: _____

Legajo: _____

Integrante 3: _____

Legajo: _____

Fecha de entrega: _____

Indice

(Click derecho > Actualizar campo para refrescar numeros de pagina)

1. Contexto	3
2. Requisitos Funcionales	3
2.1 Modelo de Datos	3
2.2 Operaciones con Streams API	4
2.3 Algoritmos de Búsqueda y Ordenamiento	4
2.4 Patron Strategy	5
2.5 Concurrencia	5
2.6 API REST Basic	6
3. Restricciones Técnicas	6
4. Entregables	7
5. Criterios de Evaluación	7
6. Información de Entrega	8

OBJETIVO EDUCATIVO

Dominar algoritmos de búsqueda, ordenamiento y procesamiento funcional de colecciones mediante Streams API, aplicar patrones de diseño Strategy e introducir conceptos de concurrencia y desarrollo de APIs REST.

1. CONTEXTO

Desarrollar el backend de una plataforma tipo Spotify que permita gestionar canciones, crear playlists automáticas y generar recomendaciones, aplicando programación funcional y concurrente.

2. REQUISITOS FUNCIONALES

2.1 Modelo de Datos

Clase Cancion:

0. id: UUID (String)
1. titulo: String
2. artista: String
3. album: String
4. género: String (ENUM preferible: ROCK, POP, JAZZ, ELECTRÓNICA, CLÁSICA)
5. duracionSegundos: int
6. Reproducciones: AtomicInteger (para concurrencia)
7. rating: double (0.0 - 5.0)
8. fechaLanzamiento: LocalDate

Agregar a este modelo las especificaciones necesarias para poder incorporar un **artista**, un **álbum** y una **productora**. Esto implica que el modelo tiene que poder soportar que un álbum tenga muchas canciones, una discográfica múltiples álbumes y cantantes

2.2 Operaciones con Streams API (30%)

- ☐ Filtrado compuesto: por género, artista, rango de años, rating mínimo
- ☐ Top 10 mas reproducidas: sorted(Comparator) + limit(10) + collect

Estadísticas:

9. promedio de duración por género (Collectors.averagingInt)
10. artista más popular (maxBy reproducciones)
11. distribución por décadas (Collectors.groupingBy)

- ❑ Playlist automática: seleccionar canciones que sumen exactamente X minutos (aproximación al problema de la mochila usando recursión y Streams)

2.3 Algoritmos de Búsqueda y Ordenamiento (20%)

- ❑ Búsqueda binaria por título: lista preordenada con `Comparator.naturalOrder()`
- ❑ Ordenamiento personalizado:
`Comparator.comparing(Cancion::getArtista).thenComparing(Cancion::getFechaLanzamiento).reversed()`
- ❑ Búsqueda lineal con predicados múltiples (género AND año > X AND rating > Y)

2.4 Patron Strategy (20%)

Interfaz Estrategia Recomendación:

```
List<Cancion> recomendar(List<Cancion> catalogo, Cancion base)
```

Implementaciones:

- ❑ RecomendaciónPorGénero: mismo género, ordenadas por rating
- ❑ RecomendaciónPorPopularidad: top 5 más reproducidas global
- ❑ RecomendacionDescubrimiento: menos de 1000 reproducciones, fecha reciente (< 2 años), género diferente al habitual

2.5 API REST Básica con Spring Boot

CancionController con:

12. GET `/api/canciones` - listar todas (con paginación opcional)
 13. GET `/api/canciones/{id}` - buscar por ID
 14. GET `/api/canciones/buscar?titulo=xxx&artista=yyy` - búsqueda filtrada
 15. POST `/api/canciones/{id}/reproducir` - incrementar contador
 16. Para las nuevas entidades de dominio (como **Artista**, **Álbum**, **Productora/Discográfica**, etc.) creadas en el requisito de Normalización del Modelo de Dominio, se deben implementar **Controllers** dedicados que soporten las siguientes operaciones básicas de consulta para cada una de las entidades principales definidas:
 17. GET `/api/{recurso}`: Listar todos los recursos de la entidad (ej., `/api/artistas`, `/api/albumes`, etc.), con soporte opcional para paginación.
 18. GET `/api/{recurso}/{id}`: Buscar una entidad específica por su ID y devolver sus metadatos principales.
 19. GET `/api/{recurso}/buscar?nombre=xxx`: Implementar un endpoint de búsqueda simple por el nombre o título de la entidad.
- ❑ Uso de `ResponseEntity` para códigos HTTP apropiados

3. RESTRICCIONES TÉCNICAS

- 20. Java 17+, Spring Boot 4.x
- 21. Base de datos H2 en memoria (opcional) o ArrayList para simplificar
- 22. Postman o curl para probar endpoints REST
- 23. Maven con dependencias: spring-boot-starter-web, lombok (opcional)

4. ENTREGABLES

- 1. Código fuente completo (proyecto Maven/Gradle)
- 2. Colección de Postman con 5 requests de prueba
- 3. Análisis de complejidad Big O para:
 - 24. Búsqueda binaria implementada
 - 25. Algoritmo de playlist automática
 - ☐ Captura de ejecución concurrente mostrando incremento de reproducciones

5. CRITERIOS DE EVALUACIÓN

Aspecto	Peso
Uso avanzado de Streams API	15%
Implementacion de Strategy	15%
Modelado de Objetos	35%
Algoritmos de busqueda/ordenamiento	5%
API REST funcional	25%
Análisis de complejidad	5%